

Beyond One-Shot: A Technical Guide to LLM System Design

A synthesis of 36 papers on why single-prompt LLM calls hit a ceiling and what actually works above that ceiling.

Companion to: [LLMs-In-Science.PDF](#) (paper-by-paper deck, 88 slides) and [llm-beyond-one-shot-linkedln.md](#) (accessible narrative for a general audience).

Audience: engineers and researchers using LLMs in production or experiments. Assumes familiarity with Transformer-era models, the OpenAI / Anthropic / Meta model families, RAG, and standard benchmarks (HumanEval, GSM8K, MATH, SWE-bench).

0. The thesis

A frontier LLM called with a one-shot prompt is a component, not a system. On easy tasks the component succeeds; on harder ones it produces fluent wrong answers. The literature of 2021–2026 has spent its effort finding the scaffolding — prompting structure, tool use, iterative refinement, search-and-verify, multi-agent debate, role specialization, principled alignment — that turns the component into a system that is actually reliable.

This document organizes that scaffolding into patterns and anti-patterns, with the empirical numbers that back each claim. Each cited paper is listed once in its load-bearing section; a full reference list with links is at the end.

1. Why one-shot prompts hit a ceiling

Two findings set the frame for everything below.

Hallucination is a property of the decoder, not a bug. Ji et al.'s *Survey of Hallucination in NLG* [38] unified a decade of results across summarization, dialogue, QA, translation, data-to-text, and vision-language tasks. The headline meta-finding: 25–30% of abstractive summaries contain factual errors even after fine-tuning, surface metrics (BLEU/ROUGE) correlate weakly with human faithfulness judgments, and the most reliable mitigations are *data-level* (retrieval augmentation, cleaned training data) rather than architectural. Translation: expect hallucination; design the system to *catch* it rather than assuming the base model won't produce it.

LLMs cannot reliably self-correct reasoning without external signal. Huang et al.'s ICLR 2024 re-evaluation [33] dismantled a wave of self-correction claims by showing that prior methods either leaked oracle labels, mis-accounted for inference compute, or smuggled hints into the "feedback" prompt. Under clean conditions, intrinsic self-correction (no oracle, no external verifier) is flat to negative: GPT-4 on GSM8K goes **88.0%** → **88.0% / 90.0%** across two correction rounds, Llama-2-70B on CommonSenseQA *collapses* **81.5%** → **74.5%**, and GPT-3.5 flips **74.7%** of GSM8K answers unnecessarily, most from correct to wrong. Multi-agent debate at matched inference cost is no better than plain self-consistency.

These two results together tell you where scaffolding has to come from: **external signal** (retrieval, tools, tests, verifiers, diverse samplers) — not additional self-talk.

2. Shaping the trace: Chain-of-Thought and Tree-of-Thoughts

The cheapest external signal is structural: force the model to emit intermediate work, then branch over it.

Chain-of-Thought (CoT) [83] replaced input→output few-shot exemplars with `(input, reasoning chain, output)` triples. PaLM-540B with 8-shot CoT reaches **56.9% on GSM8K** versus ~17.9% with standard few-shot — surpassing even fine-tuned GPT-3 175B plus a verifier. Critically, CoT is **emergent with scale**: below ~100B parameters it is neutral or harmful. On single-step tasks its benefit is small; on multi-step math it is decisive.

Tree-of-Thoughts (ToT) [88] generalizes CoT from a linear chain to a tree with an LLM-as-evaluator pruning partial states. On Game of 24 with GPT-4: IO prompting 7.3%, CoT 4.0%, CoT-self-consistency ~9%, **ToT (b=5) 74%** — an 18× improvement over CoT. On 5×5 Mini Crosswords, letter accuracy jumps from ~16% (IO+CoT) to ~78% (ToT). Ablation: replacing the LM evaluator with random selection collapses performance to CoT-SC levels — the evaluator is load-bearing.

Design rule: use CoT whenever the task decomposes into clear sub-steps and you have a model ≥GPT-3.5-class. Upgrade to ToT when (a) the task has branching decisions, (b) you can articulate a per-step state evaluator cheaply, and (c) the cost of the extra samples is acceptable (ToT does 10–100× more LLM calls).

3. Acting on the world: ReAct and CodeAct

Reasoning in isolation from the environment compounds hallucinations. Letting the model *act* closes that loop.

ReAct [89] alternates `Thought → Act → Observation` tokens, where `Act` calls a Wikipedia API, simulator, or shell. Few-shot ReAct with PaLM-540B lifts HotpotQA EM from CoT's 29.4 to **35.1 (ReAct→CoT-SC hybrid)**, and ALFWorld task success from ~22% (Act-only) to **~71%** using a 1-shot prompt — beating imitation-learning agents trained on 10^3 – 10^5 trajectories.

CodeAct [80] replaces text-or-JSON tool calls with *executable Python*. The LLM emits a program, a sandbox runs it, and the traceback comes back as the next observation. On the M3ToolEval multi-tool benchmark, Llama-2-70B-chat jumps from ~25% (text/JSON actions) to **~45% (CodeAct)** — a ~20-point absolute lift, driven by native control flow, native data flow between tool calls, free access to tens of thousands of Python packages, and automatic self-debug via tracebacks. The open CodeActAgent fine-tune generalizes to robot planning, text games, and model training without hurting base MMLU scores.

Design rule: if your task involves multi-step data flow between tools, emit *code*, not JSON. Code subsumes both the action format and the control flow, and gives you free debugging via exceptions.

4. Code LLMs as a foundation

Two papers define the floor for any code-generation pipeline.

Codex [13] established the pass@k metric and HumanEval (164 programming problems with hidden unit tests). Headline: Codex-12B hits **28.8% pass@1, 72.3% pass@100**; Codex-S reaches **37.7% / 77.5%**. The critical inference-time finding: sample 100 candidates, pick by unit tests → 77.5%; pick by mean log-prob → 44.5%; pick by back-translation reranker → 70.2%. In other words, **sample-and-rerank recovers most of the oracle gap** even without unit tests. This is the seed of every modern sample-verify system.

Code Llama [67] gave the open ecosystem a credible foundation: 7B/13B/34B/70B derived from Llama 2 with continued pretraining on a 500B-token code corpus, long-context fine-tuning to **100K tokens**, fill-in-the-middle support, and a permissive license. Code Llama-Python 34B hits **53.7% HumanEval pass@1 / 56.2% MBPP**; 70B-Python reaches **53% / 65%**. Code Llama-Python 7B beats vanilla Llama 2 70B on both benchmarks — the value of code-heavy continued pretraining overwhelms the value of raw parameter count for this task.

Design rule: for any code-gen component, start from a code-specialized model (Code Llama, DeepSeek-Coder, or a frontier API); fine-tune the base chat model only if you have domain data the coder models lack.

5. Iterative refinement — when it works and when it doesn't

Iteration isn't magic. It helps with the right signal and hurts with the wrong one.

Self-Refine [58] uses a single LLM in three prompted roles (generate → critique → revise) across 7 tasks. On *generation* tasks it averages ~20% absolute improvement — e.g. Sentiment Reversal **3.8 → 36.2 (+32.4)** on GPT-4, Dialogue Response **25.4 → 74.6 (+49.2)**, Constrained Generation **15.0 → 45.0 (+30.0)**. But on **math reasoning it is flat** (GSM8K: 92.9 → 93.1 on GPT-4, +0.2 absolute) — foreshadowing Huang et al.'s result [33].

Reflexion [72] goes further: after each episode, the agent writes a natural-language *reflection* on what went wrong and appends it to an episodic buffer used on subsequent episodes. GPT-4 HumanEval pass@1 rises from **80% → 91%**, ALFWorld success jumps **+22%** over ReAct. Ablation: without the memory buffer, the gain collapses — the *cross-episode text memory* is the mechanism, not generic prompting.

CodeChain [44] attacks monolithic code outputs. The loop asks the model to decompose into abstract sub-functions, extracts all emitted sub-modules, clusters them by functional similarity, picks one representative per cluster, and re-prompts. APPS pass@1 rises **+35% relative**; CodeContests pass@1 rises **+76% relative** on GPT-3.5. The key trick is *preserving useful sub-parts across revisions* rather than regenerating wholesale.

Self-Repair — the sobriety check. Olausson et al. [62] compared self-repair against *equal-compute i.i.d. sampling* on HumanEval and APPS. Main finding: under matched compute, self-repair's gains are **modest, inconsistent, and often zero or negative**; on Code Llama-13B, self-repair frequently underperforms plain sampling. The bottleneck is the *quality of feedback* — pairing GPT-3.5's repair head with GPT-4 feedback, or with *human* feedback, produces meaningful gains; same-model self-feedback is too weak.

Design rule:

- Use iterative refinement for tasks with clear generation quality cues (style, coverage, code readability).
 - Do **not** use it for multi-step reasoning without an external verifier or test.
 - Always compare against "spend that compute on more i.i.d. samples instead" — refinement must beat that baseline, not beat single-shot.
 - When refinement helps, the feedback channel is where the real signal lives; swap in stronger feedback (a bigger model, tests, types, humans) before increasing iteration count.
-

6. Sample broadly, verify tightly — Best-of-N with process rewards

The dominant scaling lever on *inference* is repeated sampling coupled with a verifier strong enough to pick the winner.

Large Language Monkeys [9] characterizes an inference scaling law: on CodeContests with Gemma-2B, coverage grows from **0.02% @1 sample** → **7.1% @10,000 samples** — tracking log-linear over four decades. On SWE-bench Lite with DeepSeek-Coder-V2 in Moatless Tools: **15.9% @1** → **56% @250 samples**, beating the Claude-Sonnet-3.5 + GPT-4o single-shot SOTA at roughly matched cost. Llama-3-8B with repeated sampling beats single-sample Llama-3-70B at matched FLOPs. **The critical caveat:** gains saturate when verification fails. Naive majority-voting or reward-model reranking plateaus after a few hundred samples even when ground-truth coverage keeps climbing. You need either (a) an execution-based verifier or (b) a process-level reward model.

Outcome reward models (GSM8K). Cobbe et al. [16] released GSM8K (8.5K grade-school math problems) and introduced the sample-and-verify paradigm: generate 100 candidates, train a separate verifier on (problem, solution, correct?) triples, pick the top-scored candidate. 6B generator + 6B verifier jumps from ~20% (fine-tuning alone) to **~44%** — matching or exceeding the 175B fine-tuning baseline at 30× fewer params. Verifiers are also data-efficient: **2,000 problems** with a verifier beats 8,000 problems via fine-tuning.

Process reward models (MATH). Lightman et al.'s *Let's Verify Step by Step* [52] collected **PRM800K** — 800,000 human step-level correct/incorrect/neutral labels on MATH solutions — and trained a Process Reward Model (PRM) that scores per-step correctness. Best-of-N over temperature-sampled candidates: **PRM reaches 78.2% on MATH** versus ~72% for a matched outcome-only ORM. The PRM's advantage widens with N because per-step supervision localizes errors that an outcome model sees only as "final answer wrong." PRM800K became the canonical dataset for process-reward work and directly informed OpenAI's o1.

Design rule:

- For math/code/proof: sample N candidates ($N \in [8, 1000]$ depending on budget), rerank with an *execution-based or process-level* verifier.
- Do not leave majority-vote as the selector on hard tasks — it saturates long before coverage does.
- A verifier trained on far less data than the generator still pays for itself; invest in the verifier, not more generator data.

7. Multi-agent debate: independent samples, different perspectives

Debate is sampling with structure — independent LLM instances, exposed to each other's reasoning, converging through text rather than a hand-rolled aggregator.

Multi-agent Debate [19] runs R rounds where each of k agents re-answers after seeing peers' previous responses. With 3 agents / 2 rounds on GPT-3.5: arithmetic **67.0** → **81.8% (+14.8)**, GSM8K **77.0** → **85.0% (+8)**, MMLU **63.9** → **71.1% (+7.2)**, biography factuality **66.0** → **73.8 (+7.8)**. Persona diversity matters: seeding agents as "professor / doctor / mathematician" versus identical copies raises MMLU from 71.1 → **74.2%**. Mixing *different base models* (ChatGPT + Bard) yields complementary gains.

ChatEval [12] applies the debate pattern to *evaluation* rather than answering. Multi-agent judging on FairEval lifts pairwise-preference accuracy **58.1** → **63.8%** (ChatGPT backbone, human inter-annotator ceiling 71.7%) and on Topical-Chat lifts Spearman ρ by +16.3% relative over G-Eval-4. Ablation: identical-persona agents erase most of the gain. The debate is not about consensus — it is about surfacing disagreements that a single judge would miss.

LLM-as-a-Judge [94] put guardrails on the LLM-as-judge paradigm itself. Zheng et al. released MT-Bench (80 curated multi-turn questions) and Chatbot Arena (crowdsourced Elo), and enumerated four biases — position bias (up to 30% first-answer preference), verbosity bias, self-enhancement bias (~10% preference for own outputs), and limited-reasoning bias on math/logic. Mitigations (swap-position averaging, reference grading, CoT for the judge) bring GPT-4's agreement with humans to **>80%** — matching inter-human agreement at ~1% the cost of expert evaluation.

Design rule:

- For eval/judging: use a multi-agent judge with *different personas*; always include position-swap to neutralize position bias; use pairwise comparison rather than absolute scoring when possible.
- For answering: debate is a cheap boost on factual/reasoning tasks if you can afford ~3×–5× inference cost per query. Tune persona diversity before tuning rounds.
- Remember [33]: same-model debate at matched compute does *not* beat self-consistency. Debate earns its cost only with persona/model diversity that produces genuine disagreement.

8. Role-specialized agent teams with structured artifacts

When tasks decompose cleanly (e.g. "write a program matching this spec"), role specialization with structured intermediate artifacts outperforms free-form multi-agent chat.

MetaGPT [32] models a software team as a pipeline of roles — Product Manager, Architect, Project Manager, Engineer, QA — each emitting *structured artifacts* (user stories, APIs, flow diagrams) rather than free-form dialogue, with publish-subscribe routing and an executable feedback loop. GPT-4 HumanEval pass@1 rises from 67.0% (vanilla) → **85.9% (MetaGPT)**; MBPP **68.1** → **87.7%**, +5.4 absolute over the next-best collaborative baseline. On 70 SoftwareDev tasks, executability scores **3.75/4** versus AutoGPT's 1.8. Ablation: removing the executable-feedback loop is the single largest drop.

CAMEL [47] introduced role-play with *inception prompts* — fixed system prompts seeding **AI user ↔ AI assistant** pairs, with explicit rules against loops and a `<CAMEL_TASK_DONE>` terminator. Its second contribution is scale: 25,000 AI-Society conversations + equivalents for Code/Math/Science, all released. LLaMA-7B fine-tuned on these datasets ("CAMEL-7B") reaches **HumanEval 14.0 / pass@10 57.9** vs base Llama-7B's 10.5 / 36.5 and Vicuna-7B's 11.0 / 42.9 — showing multi-agent-generated data transfers to smaller open models.

AutoGen [86] is the most general: every LLM app is a graph of *conversable agents* (customizable LLM backend + tool execution + optional human-in-the-loop) orchestrated through *conversation programs* (one-to-one, group chat, hierarchical). MATH benchmark (GPT-4): AutoGen's AssistantAgent+UserProxy beats vanilla GPT-4, LangChain ReAct, ChatGPT+Code Interpreter, and ChatGPT+Wolfram across six subcategories. OptiGuide case study: 430 → 100 lines of orchestration code, **3× faster user tasks**, and 8–35% fewer unsafe outputs.

Design rule:

- If agents produce code/specs/plans, enforce *structured artifact schemas* between them (JSON, Markdown with fixed sections) — free-form chat degrades fast on long horizons.
- Always include an executor (unit tests, compiler, linter) as a first-class agent — it's a stronger signal than any LLM critic.

- Prefer AutoGen-style primitives (conversable agents + conversation programs) over hand-rolled pipelines; it composes better as tasks grow.

9. Principles as alignment: constitutional AI and domain constitutions

Constitutional AI [6] replaces human harmfulness labels with a short written constitution (~16 natural-language principles) and an LLM's ability to critique and revise its own outputs against those principles. Two stages: SL-CAI (generate → critique → revise → fine-tune) and RL-CAI (train a preference model on AI-generated preference pairs, then RLHF). Result: CAI matches RLHF on harmfulness at 52B scale, is preferred over RLHF assistants at comparable helpfulness, and *does not become more evasive* as harmlessness training proceeds — the model *explains* its objections rather than curtly refusing. Ablation: chain-of-thought in the critique step improves preference-model accuracy by several points.

Why this matters for system design. The CAI pattern generalizes far beyond safety. A "constitution" is just a **declarative policy** that the model applies during critique — it scales naturally to domain-specific constitutions (e.g. UberScalar's *PRIME DIRECTIVE* isolates generation from verification; a scientific-review system could enshrine "do not judge publication merit; only validate arithmetic against hardware specs"; a customer-support bot could enshrine refund-policy bounds). The ingredients — principles, critique prompt, revision prompt, preference model — are portable.

Design rule: if you need enforceable behavioral constraints (safety, scope, factual grounding), write them as a short principles document and add a critique-then-revise stage before shipping outputs. It beats post-hoc filtering and makes the policy auditable.

10. Applied: LLMs as scientific comprehension layer

Everything above feeds into *applied* systems. Science was an early and instructive target: the inputs are dense, the stakes (correctness, citation accuracy) are high, and the evaluation is tractable.

S2ORC [54] is the unifying corpus: **81.1M** paper nodes, **8.1M** PDF-parsed full-text papers, **1.5M** LaTeX parses, ~**25B** body-text tokens — cross-discipline, with inline citation mentions and resolved bibliography links. This is the substrate for almost every science-LLM system since 2020.

Galactica [74] is the "one model for science" bet: 120B params pretrained on 48M curated papers + textbooks + SMILES + proteins, with special tokens for citations (`[START_REF]/[END_REF]`), working memory (`<work>`), and scientific modalities. Headline: **68.2% on LaTeX equations vs GPT-3's 49.0%, MATH 20.4% vs PaLM-540B's 8.8%** (at 4× fewer params for the 30B variant), new SOTA on PubMedQA (77.6%) and MedMCQA (52.9%). The paper was pulled from public demo three days after launch over factuality complaints — a cautionary tale that *domain-specialized pretraining* alone does not solve hallucination; it needs verification scaffolding.

SciFact [78] formalized scientific claim verification (1,409 expert-written claims + 5,183 abstracts): given a claim, retrieve abstracts, label SUPPORT / REFUTE / NOINFO with sentence-level rationales. The best VERISCI pipeline reaches **abstract-level F1 ≈ 46.1** with predicted rationales and **69.4** with oracle rationales. Error analysis identifies five missing capabilities — directional reasoning (higher/lower), numerical/statistical interpretation (p-values), scientific background, etc. — that frame the research agenda for scientific fact-checking.

CiteWorth [85] provides a cite-worthiness dataset at scale: **1.18M cleaned, contextualized sentences** across 10 domains, labelled for whether a sentence should cite. Paragraph-level Longformer-Ctx hits **F1 67.45** vs SciBERT's 62.06 — a 5-point gain driven specifically by paragraph context, not model capacity. Cite-worthiness also transfers as an *auxiliary task*: SciBERT + CiteWorth pretraining yields +1.8 F1 average on ACL-ARC citation-intent classification.

AutoSurvey [81] operationalizes survey-writing as a four-stage pipeline (retrieve → parallel subsection drafting with RAG → integrate/refine → multi-criterion eval). For a 64K-token survey: **citation recall 82.25% / precision 77.41%**, approaching human 86.33% / 77.78%; **73.6 surveys/hour** vs human 0.07; ~\$1.20 per 32K-token survey. Ablation: dropping retrieval crashes recall from 83.48 → 60.11 — RAG is the critical component. Downstream QA utility: readers answer topic questions at **67.6%** using AutoSurvey versus 58.4% from direct LLM — closer to the 73.6% ceiling.

ResearchAgent [4] tackles the *ideation* stage: start from a core paper, expand via citation graph, augment with an entity-centric knowledge store (50,091 papers, May–Dec 2023), then refine problem/method/experiment drafts with multiple ReviewingAgents whose criteria are *induced from human judgments*. Full system hits **4.52 / 4.28 / 4.18 out of 5** for problem/method/experiment on GPT-4, vs naive 4.20 / 4.03 / 3.92. Score climbs monotonically from ~3.75 → ~4.50 across 4 refinement rounds. Ablation: removing reference expansion costs more than removing the entity store (−0.26 vs −0.17 on Problem).

Design rules (applied science):

- Build on S2ORC or an equivalent structured corpus; don't re-parse PDFs from scratch.
- Always pair generation with retrieval for citation-bearing outputs — AutoSurvey's ablation is definitive.
- For claim verification, invest in *rationale supervision* (SciFact) and *cross-paper context* (CiteWorth paragraph encoding); sentence-only classifiers plateau.
- For ideation/critique loops, induce evaluation criteria from human judgments rather than prompting for abstract qualities like "novelty."

11. Applied: autonomous discovery — the frontier

A small, impressive set of systems show that LLMs paired with rigorous verification can produce *verifiable new knowledge*.

The AI Scientist [55] closes the full research loop on three ML templates (diffusion, transformer LM, grokking): ideation → experimental iteration via Aider → full LaTeX write-up → automated LLM reviewer. The automated reviewer (GPT-4o + 5-round self-reflection + 5-ensemble + 1-shot) hits **balanced accuracy 0.65, F1 0.57** on 500 ICLR 2022 papers versus human NeurIPS reviewers' 0.66 / 0.49 — slightly below humans on accuracy, *superhuman on F1*. End-to-end papers cost ~\$15 each; some clear the NeurIPS "Weak Accept" threshold. Failure modes (hallucinated plots, missed related-work sections) remain the open problems.

FunSearch [66] does something rarer: it produces **verifiable mathematical discoveries**. An LLM generates candidate programs that implement a heuristic; an evaluator scores them; an island-based evolutionary loop iterates on the top-scoring code. On the *cap set* problem, FunSearch found a cap set of size 512 in $n=8$ (larger than anything previously known) and raised the asymptotic lower bound on cap-set capacity from the long-standing 2.2180 (Tyrrell 2022) to **2.2202 (A(24,17))** — the largest improvement in ~20 years. On online bin-packing, FunSearch beats first-fit/best-fit uniformly and gets within **0.03%** of the theoretical lower bound on Weibull-100k. The trick is that the output is a *program*, so discoveries are concise, interpretable, and verifiable.

AlphaGeometry [75] is a neuro-symbolic prover: an LLM proposes auxiliary constructions (new points/lines) and a symbolic deduction engine (DD+AR) verifies and extends. Trained from scratch on **100M synthetic theorem-proof pairs** generated by a symbolic pipeline (no human proofs). On the IMO-AG-30 benchmark: **AlphaGeometry 25/30** vs DD+AR alone 14/30 vs GPT-4 **0/30** on full natural-language proofs. The average IMO gold medallist solves 25.9/30 — AlphaGeometry is at the gold-medal bar.

Formal Math Statement Curriculum Learning [63] tackles Lean theorem-proving with expert iteration on a 774M GPT-f: generate proof-search trajectories, retain successful ones, retrain. With a miniF2F-adjacent synthetic inequality curriculum: **miniF2F-valid pass@1 = 33.6%, pass@64 = 47.3%**, beating PACT by +9.7 points. The insight: the *curriculum of problem statements* substitutes for the self-play curriculum that games get for free.

LeanDojo [87] opens the toolchain: 98,734 theorems, 217,776 tactics, 129,243 premises extracted from Lean + mathlib, plus a programmatic API for interactive proof search. **ReProver** — the first retrieval-augmented LLM tactic generator — proves **51.2%** of LeanDojo benchmark theorems vs 47.6% non-retrieval T5 vs GPT-4 zero-shot 29.0%, trained in 5 days on a single GPU. The **novel_premises** split deliberately exposes memorization; retrieval is load-bearing on it.

Design rules (discovery):

- Verifiable discovery requires a closed verifier: programs that can be tested, proofs that a solver can check, or measurements that a simulator can run. Without that, generation plus critique produces plausible nonsense.
- Structure the search space so the LLM evolves a *program* (FunSearch) or a *tactic* (LeanDojo) rather than a raw answer — artifacts compose; answers don't.
- Synthetic pretraining curricula can substitute for scarce human data ([75, 63]) — but only if the synthetic distribution matches the test distribution.

12. Applied: automated peer review — useful, not sufficient

Can LLMs provide useful feedback? [50] Liang et al. ran GPT-4 on 3,096 Nature papers + 1,709 ICLR papers and measured comment overlap with human reviewers. GPT-4 ↔ individual reviewer overlap: **30.85% (Nature) / 39.23% (ICLR)** — comparable to inter-human reviewer overlap (28.58% / 35.25%). Shuffle controls collapse to 3.43%, so the signal is paper-specific. Prospective study (308 researchers, 110 institutions): **57.4% find LLM feedback helpful, 82.4% find it more useful than at least some human reviewer, 50.5% would re-use it**. The failure mode: GPT-4 over-indexes on "add more experiments/datasets" and under-indexes on deep methodological critique.

How prevalent is LLM use in papers now? [51] The same group built a population-level mixture-proportion estimator (prediction error $\leq 3.5\%$) and ran it on 950K papers (Jan 2020 – Feb 2024). By Feb 2024, **~17.5% of CS abstracts are LLM-modified** (up from near-zero pre-November 2022); EE/SS ~13.1%, bioRxiv ~6.3%, Math lowest at ~3.5%. Heavy preprint posters (≥ 3 /year) reach ~19.7% — production pressure correlates with LLM use.

Design rules:

- LLM review complements, not replaces, human review. Use it to surface surface-level issues cheaply and to give authors fast feedback; keep human reviewers for methodological judgment.

- Measure the distributional prevalence of LLM-generated content in your corpus before training on it ([51]'s estimator generalizes).

13. Applied: multi-agent scientific illustration

PaperBanana [95] applies the multi-agent pattern to figure generation: Retriever (fetches reference illustrations), Planner (structured content plan), Stylist (style guidelines), Visualizer (VLM or Matplotlib code), Critic (compares against source intent, refines — 3 rounds). Backbones: Gemini-3-Pro for planning/critique, Nano-Banana-Pro for images. On **PaperBananaBench** (292 NeurIPS 2025 methodology diagrams), the system hits Overall ~**50.2** vs the best baseline ~43.2 — **+17% relative**, with +37.2% on conciseness and +12.9% on readability. Judge-human agreement: Spearman 0.51–0.60.

This is a clean demonstration that the multi-agent + iterative-refinement + critic-loop pattern (Sections 7–8) transfers from text to visual artifacts.

14. Design rules, consolidated

Scaffolding hierarchy (use in order of leverage, not effort):

1. **Add external signal.** Retrieval, tools, unit tests, compilers, simulators, verifiers. Every other pattern degrades to self-talk without this.
2. **Shape the trace.** CoT on anything multi-step; ToT when the task branches and you have a state evaluator.
3. **Let the model act.** ReAct/CodeAct for anything environment-coupled. Prefer code over JSON for action formats.
4. **Sample broadly, verify tightly.** Best-of-N with an execution-based or process-reward verifier beats refinement on hard reasoning. Always budget sampling vs. refinement at matched compute.
5. **Iterate on strong signal, not weak.** Refinement helps when feedback carries real information (tests, human critique, stronger-model critique). Same-model self-critique is usually a waste of tokens.
6. **Diversify before you iterate.** Multi-agent debate with *different personas or models* produces disagreement that converges on truth; identical agents produce consensus that amplifies errors.
7. **Specialize roles with structured artifacts.** For multi-step production workflows, enforce schemas between agents and include an executor/tester as a first-class role.
8. **Write the constitution.** Declarative principles + critique-then-revise scales better than post-hoc filters and makes policy auditable.

Anti-patterns:

- Single-prompt generation for multi-step reasoning or code tasks — you're leaving order-of-magnitude accuracy on the table.
- Self-correction without external signal — [33] and [62] are definitive.
- Majority-vote selection on hard reasoning — coverage keeps climbing long after majority-vote saturates [9].
- Free-form multi-agent chat for production workflows — degrades to loops and hallucinated specs [32].
- LLM-as-judge without swap-position averaging and persona diversity — bakes in position and self-enhancement biases [94].
- Fine-tuning a chat model when a code-specialized model already exists for the task [67].

References

Organized by section of first appearance. Links point to arXiv or the paper's canonical venue.

§1 — Ceiling

- [38] Ji et al., *Survey of Hallucination in Natural Language Generation* — ACM Computing Surveys vol. 55 — <https://arxiv.org/abs/2202.03629>
- [33] Huang et al., *Large Language Models Cannot Self-Correct Reasoning Yet* — ICLR 2024 — <https://arxiv.org/abs/2310.01798>

§2 — Prompt structure

- [83] Wei et al., *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models* — NeurIPS 2022 — <https://arxiv.org/abs/2201.11903>
- [88] Yao et al., *Tree of Thoughts: Deliberate Problem Solving with Large Language Models* — NeurIPS 2023 — <https://arxiv.org/abs/2305.10601>

§3 — Reasoning + acting

- [89] Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models* — ICLR 2023 — <https://arxiv.org/abs/2210.03629>
- [80] Wang et al., *Executable Code Actions Elicit Better LLM Agents (CodeAct)* — ICML 2024 — <https://arxiv.org/abs/2402.01030>

§4 — Code foundations

- [13] Chen et al., *Evaluating Large Language Models Trained on Code (Codex)* — 2021 — <https://arxiv.org/abs/2107.03374>
- [67] Rozière et al., *Code Llama: Open Foundation Models for Code* — 2023 — <https://arxiv.org/abs/2308.12950>

§5 — Iterative refinement

- [58] Madaan et al., *Self-Refine: Iterative Refinement with Self-Feedback* — NeurIPS 2023 — <https://arxiv.org/abs/2303.17651>
- [72] Shinn et al., *Reflexion: Language Agents with Verbal Reinforcement Learning* — NeurIPS 2023 — <https://arxiv.org/abs/2303.11366>
- [44] Le et al., *CodeChain: Modular Code Generation Through Chain of Self-Revisions* — 2024 — <https://arxiv.org/abs/2310.08992>
- [62] Olausson et al., *Is Self-Repair a Silver Bullet for Code Generation?* — ICLR 2024 — <https://arxiv.org/abs/2306.09896>

§6 — Sample-and-verify

- [9] Brown et al., *Large Language Monkeys: Scaling Inference Compute with Repeated Sampling* — 2024 — <https://arxiv.org/abs/2407.21787>
- [16] Cobbe et al., *Training Verifiers to Solve Math Word Problems (GSM8K)* — 2021 — <https://arxiv.org/abs/2110.14168>
- [52] Lightman et al., *Let's Verify Step by Step* — ICLR 2024 — <https://arxiv.org/abs/2305.20050>

§7 — Multi-agent debate / LLM-as-judge

- [19] Du et al., *Improving Factuality and Reasoning in Language Models through Multiagent Debate* — ICML 2024 — <https://arxiv.org/abs/2305.14325>
- [12] Chan et al., *ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate* — 2023 — <https://arxiv.org/abs/2308.07201>
- [94] Zheng et al., *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena* — NeurIPS 2023 — <https://arxiv.org/abs/2306.05685>

§8 — Role-specialized frameworks

- [32] Hong et al., *MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework* — ICLR 2024 — <https://arxiv.org/abs/2308.00352>
- [47] Li et al., *CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society* — NeurIPS 2023 — <https://arxiv.org/abs/2303.17760>
- [86] Wu et al., *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation* — 2023 — <https://arxiv.org/abs/2308.08155>

§9 — Constitutional AI

- [6] Bai et al., *Constitutional AI: Harmlessness from AI Feedback* — 2022 — <https://arxiv.org/abs/2212.08073>

§10 — Science comprehension

- [54] Lo et al., *S2ORC: The Semantic Scholar Open Research Corpus* — ACL 2020 — <https://arxiv.org/abs/1911.02782>
- [74] Taylor et al., *Galactica: A Large Language Model for Science* — 2022 — <https://arxiv.org/abs/2211.09085>
- [78] Wadden et al., *Fact or Fiction: Verifying Scientific Claims (SciFact)* — EMNLP 2020 — <https://arxiv.org/abs/2004.14974>
- [85] Wright & Augenstein, *CiteWorth: Cite-Worthiness Detection for Improved Scientific Document Understanding* — Findings ACL-IJCNLP 2021 — <https://arxiv.org/abs/2105.10912>
- [81] Wang et al., *AutoSurvey: Large Language Models Can Automatically Write Surveys* — 2024 — <https://arxiv.org/abs/2406.10252>
- [4] Baek et al., *ResearchAgent: Iterative Research Idea Generation over Scientific Literature* — 2024 — <https://arxiv.org/abs/2404.07738>

§11 — Autonomous discovery

- [55] Lu et al., *The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery* — 2024 — <https://arxiv.org/abs/2408.06292>
- [66] Romera-Paredes et al., *Mathematical Discoveries from Program Search with Large Language Models (FunSearch)* — Nature 625:468–475, 2024 — <https://www.nature.com/articles/s41586-023-06924-6>
- [75] Trinh et al., *Solving Olympiad Geometry Without Human Demonstrations (AlphaGeometry)* — Nature 625:476–482, 2024 — <https://www.nature.com/articles/s41586-023-06747-5>
- [63] Polu et al., *Formal Mathematics Statement Curriculum Learning* — 2022 — <https://arxiv.org/abs/2202.01344>
- [87] Yang et al., *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models* — NeurIPS 2023 — <https://arxiv.org/abs/2306.15626>

§12 — Peer review

- [50] Liang et al., *Can Large Language Models Provide Useful Feedback on Research Papers?* — 2024 — <https://arxiv.org/abs/2310.01783>
- [51] Liang et al., *Mapping the Increasing Use of LLMs in Scientific Papers* — 2024 — <https://arxiv.org/abs/2404.01268>

§13 — Scientific illustration

- [95] Zhu et al., *PaperBanana: Automating Academic Illustration for AI Scientists* — 2026 — <https://arxiv.org/abs/2601.23265>

This document complements [LLMs-In-Science.PDF](#) (which walks each paper in detail with speaker notes) and [Llm-beyond-one-shot-Linkedin.md](#) (an accessible narrative version for a general professional audience).